

A first-person perspective from a motorcycle rider. The central focus is the motorcycle's instrument cluster, featuring a large circular speedometer with a needle pointing to approximately 40 mph. Below the speedometer are several smaller gauges and indicator lights. The handlebars and mirrors are visible on either side. The background is a blurred asphalt road stretching into the distance, flanked by green grass and foliage, conveying a sense of high speed.

Asymptotic Analysis

Hithesh Vurjana



**If you can't measure it, you
can't improve it.**

Peter Drucker

**You can't manage what you
can't measure.**

Peter Drucker



What is the problem?

1. How algo runtime grows as input size grows?
2. How much space will it take?
3. Is the problem solvable?
4. How efficient is my code or algorithm?
5. How to reduce lag in software?
6. How to explain growth rate of an algo?
7. How to identify computational resources needed by an algo?



Techniques

1. Different ways of measuring algo runtime
2. Speed and memory usage



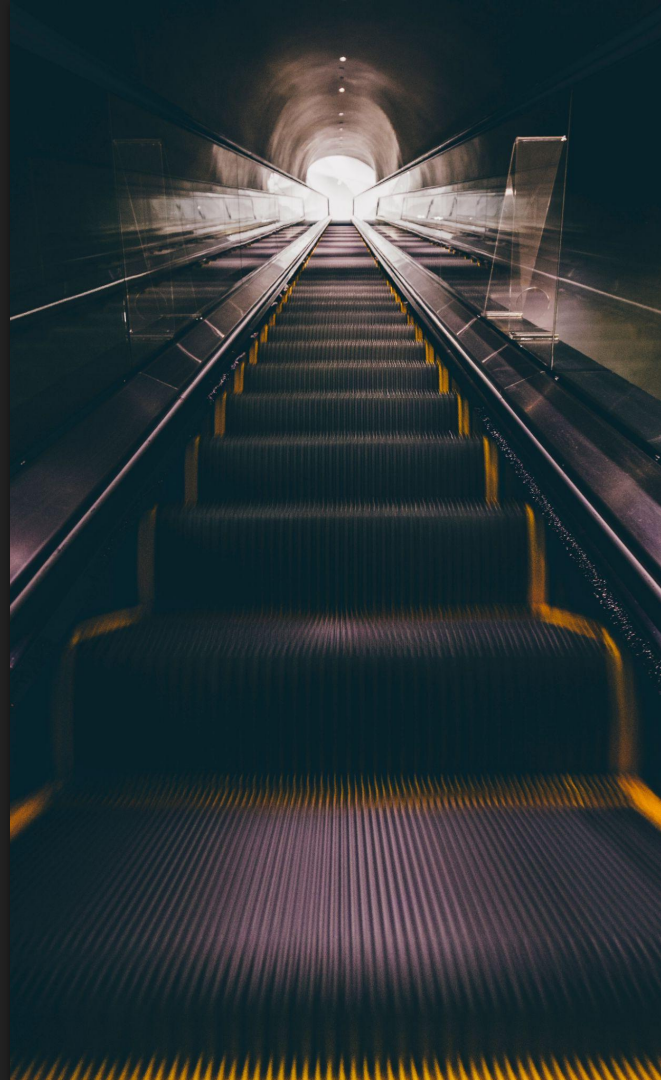
Asymptotic Analysis as a solution

1. a.k.a asymptotics
2. Method of describing limiting behavior.



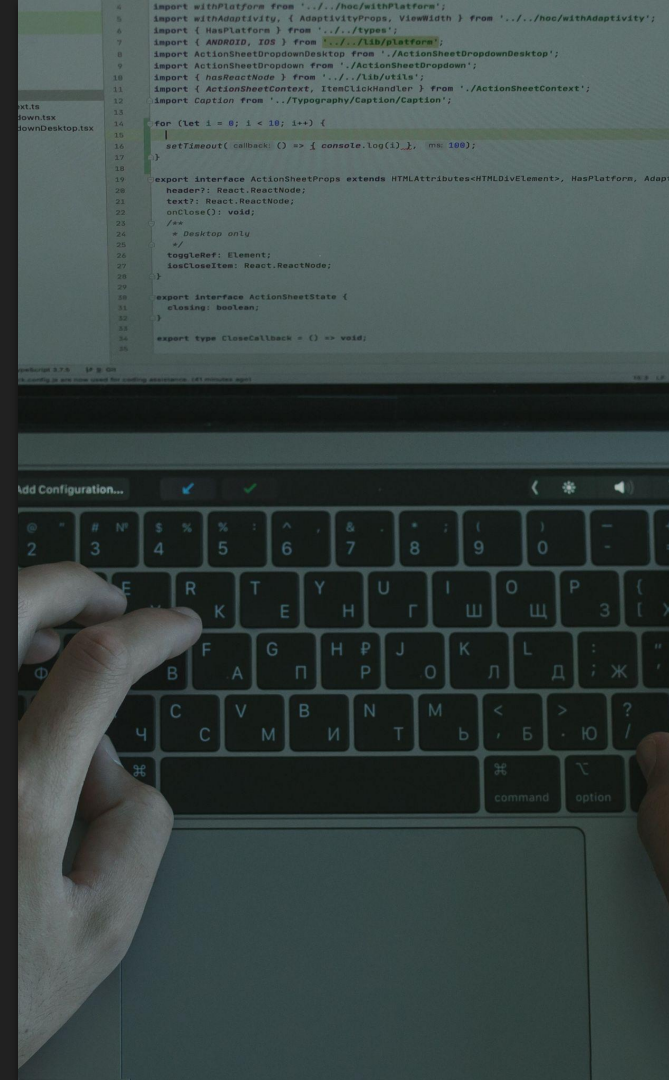
What is an Algorithm?

1. Step by step procedure for solving a computational problem.
2. Design concept.
3. Independent of programming language.
4. Implementation details don't matter.



What is a program?

1. It's about Details
2. It's done after designing the algorithm.
3. Basically implementing the algo in code.



Algorithm vs Program

1. Algo is design time while Program is at implementation time.
2. Algo is written in any language while Program is specific to particular language.



Why is it called Time Complexity?

1. Where did the name come from? Why complexity?
2. It's not a tool to tell precisely how long an algorithm will run?
3. It tells us the growth rate of an algorithm w.r.t input size, not exact time an algo took.



Time & Space Complexity

1. Time complexity != Time taken to run program
2. It's about **efficiency** of algorithm
3. Rate of growth of time/space w.r.t input
4. Assume all are using same spec computer
5. A 20TB file sent digitally vs physically
6. Digital transfer $O(N)$ - Linear time
7. Physical transfer $O(1)$ - Constant time



8. Time complexity is a math function
9. Math func that tells us how time grows as input grows
10. Time complexity tells us the relationship btw time and the input size in algo
11. Always ignore small numbers,
12. $O(1) < O(\log N) < O(N)$ - Binary Search better than Linear Search



- 13. Time complexity for large data
- 14. We ignore all constants as they are negligible
- 15. Always look at worst case with large amount of data, ignore constants
- 16. In $O(N^3 + \log N) \Rightarrow F(1m) = (1m)^3 + 6 \text{ sec}$. 6 sec is nothing in front of $1m^3$ hence ignore
- 17. Big O is the upper bound. It can be less but does not exceed the derived complexity.



- 18. Divide and conquer recurrence relationship
- 19. Basic Integration
- 20. Big O is like rating system for algorithm
- 21. Big O informs us of resources we might need for algo to run as the data increases
- 22. It informs us the resource needs of an algo
- 23. A good way to visualise is by a graph curve



- 24. Big O tells us the growth rate of the increase of resources as the data grows. Its like - how much fuel we need to provide the car for it to run as the distance to the destination gets longer. We are looking for rate of growth. Best described by calculus. Change of something.
- 25. Big O - Informs us, how runtime changes as input get bigger - This is what Big O time means.
- 26. Explain rate of growth of time by how many units?



27. Like if the input increases by say 10 elements in an array then the time increases by 2 sec with each iteration. Something as precise as this. Show the rate of growth of an algorithm by x amount of time for N amount of input at each iteration.
28. What does input getting larger mean? How input becomes bigger with each iteration? How input size dynamically increases in algorithm?



How Complexity is calculated?

1. Asymptotic Analysis is the study of growth of algorithm w.r.t input size.
2. Has nothing to do with exact time of an algorithm.
3. It is independent of any type of computer whether fast or slow.
4. An algo can take 1 min on slow computer and 1 hr on another but the algo behaviour remains same across all computers.

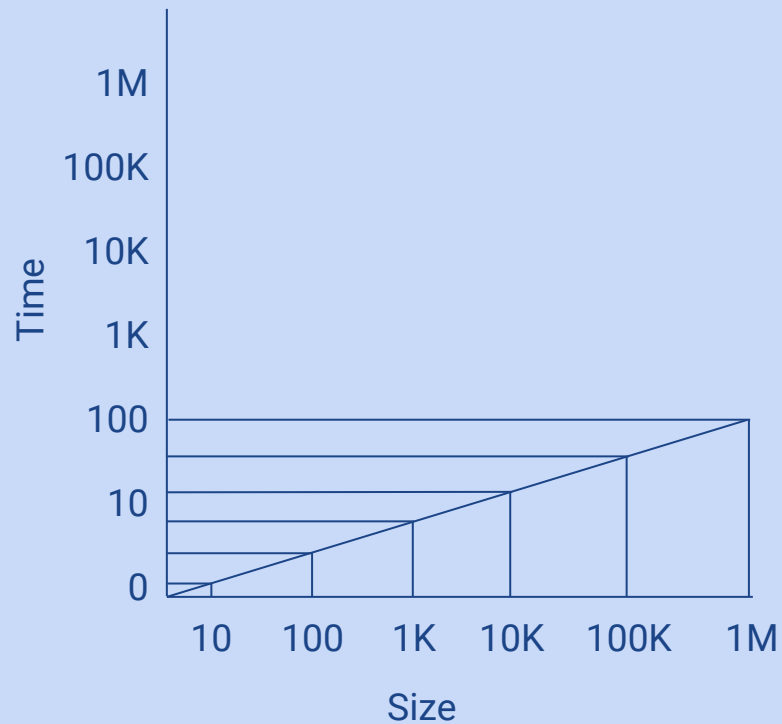
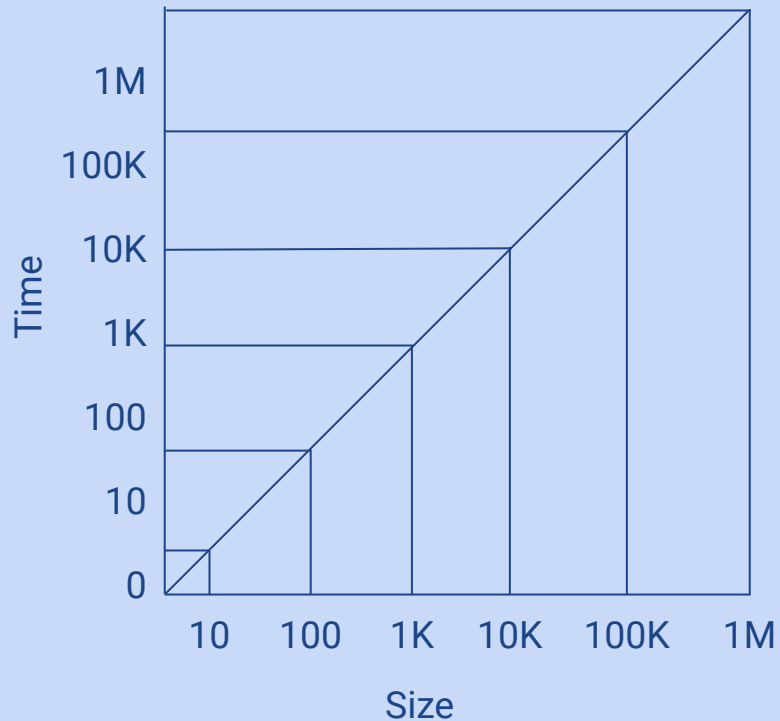


Why constants are ignored?

1. Constants are ignored as we are not concerned about the exact time of the algorithms.
2. Also constants are generally very small compared to the size of N .
3. Coefficients are ignored for the very same reason.



Apple 1 vs Mac Pro M1



Code Time: Calculate exact time of an algorithm

```
fun calculateRunTime(): Long {  
    val start = System.currentTimeMillis()  
    myFunction()  
    val end = System.currentTimeMillis()  
    return end - start  
}
```

- This running time is irrelevant as we only care about relationship btw time and input size.
- Above code tells you the exact time an algorithm takes on your machine.

Manual Run-Time Calculation

1. Each statement will take x amount of time to execute.
2. “ x ” can be any amount of time. 1 milli sec, 1 sec, 1 min, etc.

```
fun print1234() {  
    print("1")  
    print("2")  
    print("3")  
    print("4")  
}
```

Asymptotic Notations

1. Big O - Worst case
2. Big Theta - Average case
3. Big Omega - Best case



Common Big O Runtimes

1. $O(1)$ - Constant Time / Space
2. $O(\log N)$ - Logarithmic Time / Space
3. $O(N)$ - Linear Time / Space (N is input size)
4. $O(\sqrt{N})$
5. $O(N \log N)$ - Super Logarithmic Time / Space
6. $O(AB)$ - 2 different inputs
7. $O(N^2)$ - Quadratic Time / Space

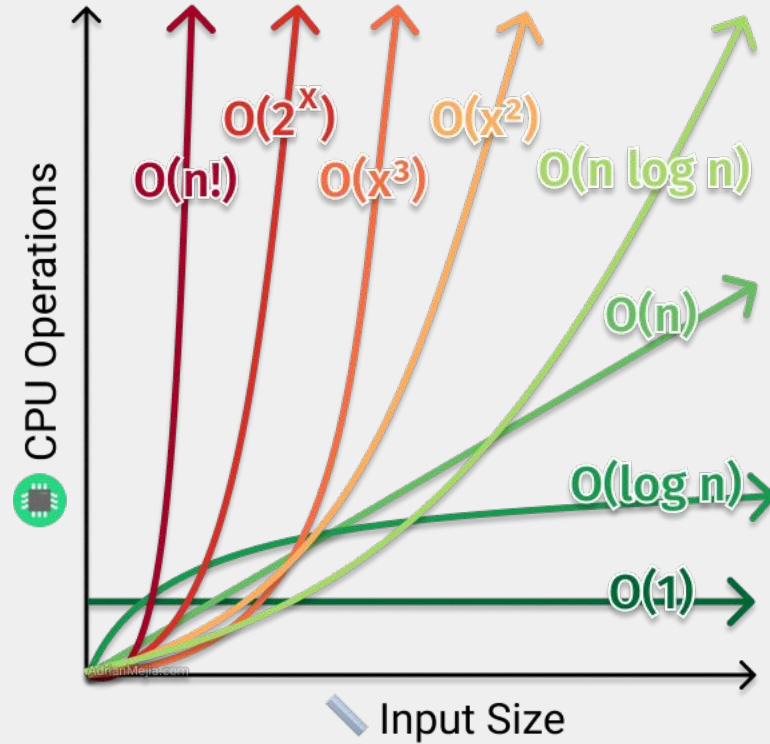


8. $O(N^c)$ - Polynomial Time / Space (c is constant)
9. $O(c^N)$ - Exponential Time / Space
10. $O(N!)$ - Factorial Time / Space
11. $O(N^N)$
12. $O(2^x * N! *)$





Time Complexity



1. $O(1)$ - Constant Time / Space

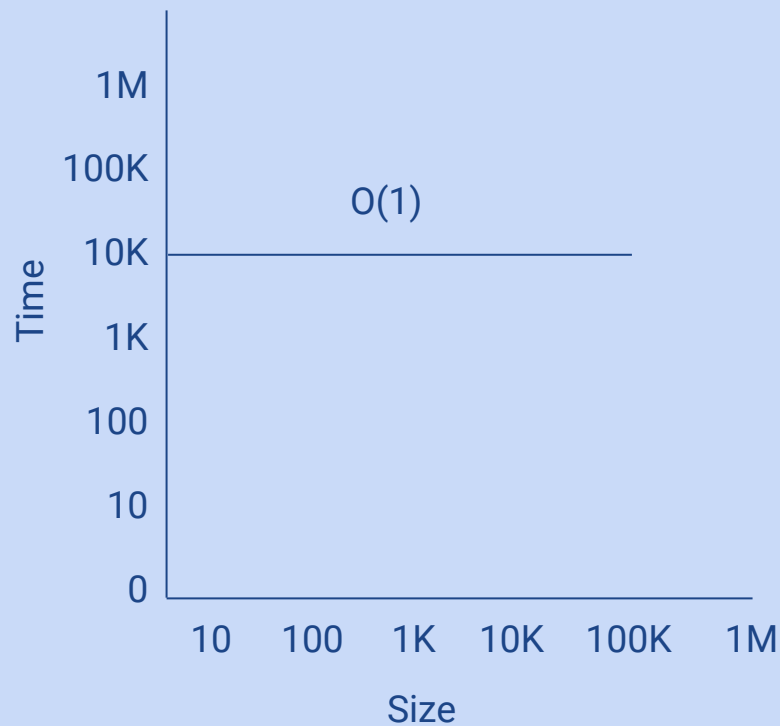
1. Even if the input size grows at every iteration the time taken by the algorithm remains the same.
2. No additional data structures used.



Code Time: $O(1)$ time & space

```
fun divisibility(n: Int) {  
    if (n % 2 == 0) println("Even") else println("Odd")  
}
```

$O(1)$ Graph



1. $O(N)$ - Linear Time / Space

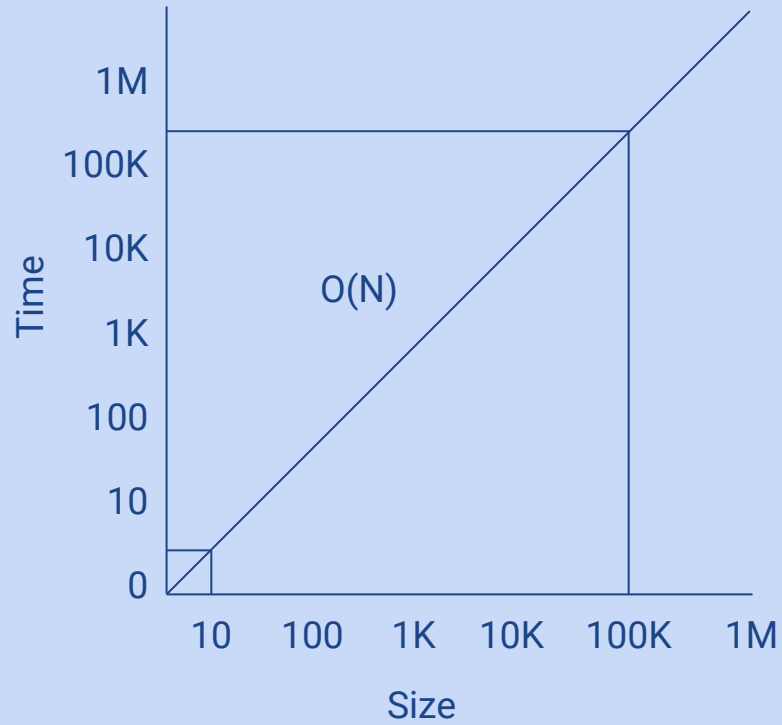
1. Time taken by the algorithm increases by a single unit of time on each iteration.
2. It means a data structure of size N is used additionally to hold the state.



Code Time: $O(N)$ time & space

```
fun pairSum(a: Int, b: Int): Int = a + b
fun pairSumSequence(n: Int): Int {
    var sum = 0
    for (i in 0 until n) sum += pairSum(i, b: i + 1)
    return sum
}
```


$O(N)$ Graph



Recap

- On Measurement
- Asymptotic Analysis?
- Algorithm Vs Program
- Time Complexity?
- Space Complexity?
- Big O
- Big Theta
- Big Omega
- $O(1)$ time / space
- $O(\log N)$ time / space
- $O(N)$ time / space
- $O(N^c)$ time / space
- $O(c^N)$ time / space
- $O(N!)$ time / space

**Controlling
complexity is the
essence of
computer
programming.**

Brian Kernighan



References

1. Cracking the Coding Interview Book - [Link](#)
2. Algorithmic efficiency - [Link](#)
3. Introduction to Algorithms - [Link](#)
4. Many more...

Thank You

